VR 人形机器人遥操作项目阶段路线

本文档按照项目发展的时间顺序，记录从原始 HTS 手部追踪测试，到脚部和全身关节扩展，再到数据录制、离线回放、机器人模拟器接入和 VR 视觉回传的完整路线。文档的写法采用“一个阶段接一个阶段”的形式，不单独区分已完成和未完成部分，方便作为项目整体规划和开发记录使用。

项目最终目标：

```
机器人视觉/状态 -> VR 设备显示
VR 用户头部、手部、身体动作 -> 控制机器人
```

也就是构建一个面向人形机器人的 VR 遥操作系统。

1. 总体系统结构

系统由两条数据流组成。

机器人到 VR：

```
机器人相机 / 深度传感器 / 状态信息
        ↓
PC 或机器人端编码与转发
        ↓
Unity / Quest 接收
        ↓
VR 中显示机器人第一视角、状态 HUD 或深度信息
```

VR 到机器人：

```
Quest 获取用户头部、手部、身体动作
        ↓
Unity BodyLandmarkStreamer 输出全身关节
        ↓
Python 中间层接收、记录、可视化
        ↓
坐标统一、人体特征预处理、学习式 retargeting、策略控制、安全限制
        ↓
机器人模拟器
        ↓
真实机器人
```

系统中每一层的职责需要保持清晰：

- Unity/Quest 层负责采集 VR 设备可获得的人体数据。
- Python 接收层负责把 Unity 发来的文本数据解析成结构化数组。
- Python 可视化和录制层负责检查数据是否稳定、连续、方向正确。
- 学习式 retargeting 层负责从人体骨架中生成机器人参考姿态。
- 机器人控制层负责策略推理、限位、速度约束和安全保护。
- VR 显示层负责把机器人视觉和状态回传给用户。

整体开发原则：

- 先验证数据，再控制机器人。
- 先离线回放，再实时控制。
- 先检查头部、双手、骨盆和双脚等关键点，再扩展到完整全身特征。
- 先在模拟器中验证，再接真实机器人。
- 每一步都保留可视化和日志，避免黑箱调试。

整体阶段分为三条主线：

```
A. HTS 数据链路：采集、可视化、录制、回放和质量检查。
B. 模拟学习训练：基于录制数据，在模拟器中训练 retargeter 和 TWIST2-style policy。
C. Sim2Real 与最终闭环：模拟器完整验证后，迁移到真实机器人，并完成 VR 视觉反馈闭环。
```

推荐推进顺序：

```
1. 先完成 HTS 数据链路，保证 84 关节人体数据可信、可录制、可回放。
2. 然后进入模拟学习训练，用录制数据训练人体动作到机器人参考姿态的 retargeter。
3. 在模拟器中重新训练适配当前 VR 设备和当前机器人的 TWIST2-style policy。
4. 完成离线 recording 和实时 VR 输入下的模拟器闭环验证。
5. 最后进入 sim2real，把通过模拟验证的系统迁移到真实机器人。
6. 真实机器人稳定后，再加入机器人视觉回传，形成最终闭环 VR 遥操作。
```

阶段去重说明：

```
阶段 1 是阶段 2 的前置验证，不是长期独立模块。
阶段 5 是阶段 4 的增强版录制工具，不是新的算法阶段。
阶段 9 和阶段 10 可以在代码中合成同一个 preprocessing pipeline，但文档中分开写，方便先检查关键点选择，再检查坐标统一。
阶段 18 是阶段 19 的子系统；真实机器人控制稳定前可以暂缓，但最终闭环必须包含它。
```

去重后的主执行视图：

```
A. HTS 数据链路
原始手部链路验证
全身 84 关节输出
实时可视化和坐标检查
录制数据，并在录制时同步可视化
离线回放和数据质量检查

B. 模拟学习训练
定义训练问题
构建学习数据集
预处理：关键点/时序特征/坐标统一/归一化
接入机器人模型和训练模拟器
生成 teacher label / robot reference
训练 learned retargeter
重新训练 TWIST2-style policy
离线 recording 模拟闭环
实时 VR 模拟闭环

C. Sim2Real 与最终闭环
真实机器人分阶段部署
机器人视觉回传
最终闭环 VR 遥操作
```

各主线的输入、输出和完成标准：

```
A. HTS 数据链路
输入：Quest / VR 设备中的人体 tracking 数据
输出：可信的 recordings/*.npz 和可视化/回放工具
完成标准：84 关节顺序、左右方向、头部方向、时间戳和数据连续性都可以被离线复查

B. 模拟学习训练
输入：多段已检查的 HTS recording
输出：learned retargeter checkpoint、robot reference/mimic obs、重新训练的 TWIST2-style policy checkpoint
完成标准：离线 recording 和实时 VR stream 都能在模拟器中稳定驱动机器人，不直接把人体坐标当机器人姿态

C. Sim2Real 与最终闭环
输入：已经通过模拟验证的 retargeter 和 policy
输出：真实机器人安全遥操作系统，以及机器人视觉/状态回传到 VR
完成标准：真实机器人分阶段执行稳定，有急停、限位、断连保护和端到端延迟监控
```

关键阶段边界：

```
阶段 0-2：只回答“VR 设备能不能稳定给出人体数据”
输入：Unity/Quest tracking
输出：手部、腿部、全身 84 关节协议
不做：机器人映射、训练、控制

阶段 3-6：只回答“这些人体数据是否可信、可记录、可复查”
输入：实时 84 关节 stream
输出：可视化结果、recordings/*.npz、离线质量报告
不做：把人体动作解释成机器人动作

阶段 7-10：只回答“学习问题和学习输入是否定义清楚”
输入：已检查的 recordings/*.npz
输出：dataset split、learning_features、coordinate calibration、normalization statistics
不做：训练 policy 或连接真实机器人

阶段 11-12：只回答“机器人模型和训练标签是否准备好”
输入：机器人模型、learning_features、动作片段
输出：可播放的 robot reference、teacher qpos、mimic obs、质量过滤结果
不做：把 teacher 当最终控制方法

阶段 13-14：只回答“模型和策略是否学会了这套任务”
输入：learning_features、teacher/reference、机器人训练环境
输出：retargeter checkpoint、TWIST2-style policy checkpoint、训练日志和评估结果
不做：真实机器人部署

阶段 15-16：只回答“模拟闭环是否真的跑通”
输入：recording 或实时 VR stream、retargeter、policy
输出：稳定的离线模拟和实时模拟闭环
不做：绕过模拟直接上真实机器人

阶段 17-19：只回答“能否安全迁移到真实机器人并形成最终 VR 闭环”
输入：模拟验证通过的 retargeter 和 policy
输出：真实机器人遥操作、视觉回传、最终闭环系统
不做：在真实机器人上临时探索未经模拟验证的动作
```

2. 阶段 0：原始 HTS 手部追踪验证

目标：确认原始 hand-tracking-streamer 项目可以运行，并建立最基础的数据链路。

这个阶段的核心是不要立刻修改功能，而是先确认原始项目的手部数据从 Quest 到 Python 的链路是通的。只有原始手部链路稳定，后续扩展脚部和全身才有可靠基础。

工作内容：

- 打开原始 Unity HTS 项目。
- 确认 Unity 项目可以正常编译。
- 在 Quest 中启用手部追踪权限。
- 运行 Unity 场景或构建到 Quest。
- 确认 Unity 中的 hand streamer 能输出左右手 wrist 和 landmarks。
- 在 PC 端运行 socket 接收脚本。
- 通过 TCP 或 UDP 接收 Unity 发来的文本消息。
- 使用原始 `visualizer.py` 显示左右手关节。

相关脚本：

```
scripts/sockets.py
scripts/visualizer.py
```

典型 TCP 调试命令：

```
adb reverse tcp:8000 tcp:8000
python scripts/sockets.py --protocol tcp --host localhost --port 8000
```

典型 UDP 调试命令：

```
python scripts/sockets.py --protocol udp --host 0.0.0.0 --port 9000
```

典型可视化命令：

```
python scripts/visualizer.py --protocol tcp --host localhost --port 8000 --show-fingers
```

需要确认的数据内容：

```
Left wrist
Right wrist
Left landmarks
Right landmarks
```

需要观察的现象：

- 左手移动时左手点云移动。
- 右手移动时右手点云移动。
- 手腕位置和手指 landmarks 大致连续。
- TCP/UDP 消息没有明显中断。
- Python 可视化窗口不会频繁卡死。

常见问题：

- Unity 编译失败时，先看 Console 中的 C# compile error。
- TCP 没有数据时，先确认 `adb reverse tcp:8000 tcp:8000` 是否执行。
- UDP 没有数据时，检查 Quest 和 PC 是否在同一网络，以及防火墙是否阻止端口。
- Python 缺依赖时，安装 `numpy` 和 `matplotlib`。

检查点：

```
Quest / Unity -> TCP/UDP -> Python -> 手部可视化
```

3. 阶段 1：脚部和腿部关节扩展

目标：在原始手部追踪基础上，尝试获取脚部和腿部相关关节。

这个阶段的关键认识是：脚部数据不是 Hand Tracking 的扩展，而是 Meta Body Tracking 的一部分。因此 Unity 中不能只寻找“脚部 tracking”选项，而需要启用 Body Tracking，并使用 `Body` 组件提供的 FullBody skeleton。

工作内容：

- 新增 `FootLandmarkStreamer.cs`。
- 在 Unity 中启用 Meta Body Tracking。
- 在 Oculus/Meta runtime settings 中启用 Full Body joint set。
- 确认应用启动时请求 Body Tracking 权限。
- 将 foot streamer 挂载到带有 `Body` 组件的 GameObject。
- 避免把 streamer 同时挂在 `OVRBody` 和 `OVRBodyDataSource` 两处。
- 通过 Unity 日志确认 Body Tracking 状态。
- 输出脚踝、膝盖、脚跟、脚尖等脚部/腿部相关关节。
- 在 Python 端扩展脚部数据解析和可视化逻辑。

Unity 层级检查原则：

```
streamer 应挂载在有 Body 组件的对象上。
数据源对象只负责提供数据，不应重复挂同一个 streamer。
```

Body Tracking 需要检查的状态：

```
Connected
Valid
HighConf
UpdatedEvent
FullBodyFoot
```

典型日志目标：

```
Connected=True
Valid=True
HighConf=True
UpdatedEvent=True
FullBodyFoot=True
```

脚部阶段常用的关节目标：

```
LeftKnee
RightKnee
LeftAnkle
RightAnkle
LeftFootHeel
RightFootHeel
LeftFootBall
RightFootBall
```

Python 可视化需要观察：

- 左脚和右脚没有交换。
- 膝盖、脚踝、脚尖的顺序合理。
- 抬左腿时左侧腿部数据变化。
- 抬右腿时右侧腿部数据变化。
- 脚部点不是固定不动。
- 数据更新频率和手部类似。

关键结论：

```
脚部不是独立的 Hand Tracking 功能，而是来自 Meta Body Tracking 的 FullBody skeleton。
```

常见问题：

- `Connected=True` 但 `UpdatedEvent=False` 时，通常需要检查 `OVRBodyDataSource` 和 `Body` 的连接关系。
- 同一对象层级中挂了多个 streamer 时，日志可能出现重复或左右混乱。
- Quest 没有给 Body Tracking 权限时，Unity 端可能能运行但没有有效数据。
- 脚部数据本身是推断结果，稳定性通常弱于手部。

检查点：

```
Quest / Unity -> FootLandmarkStreamer -> Python -> 脚部和腿部可视化
```

4. 阶段 2：全身 84 个关节输出

目标：从脚部扩展到完整全身关节输出。

这个阶段不再只针对脚部，而是把 Meta FullBody skeleton 中的所有关节统一输出。这样后续机器人映射时可以自由选择头、肩、肘、手腕、骨盆、膝盖、脚踝等关键点，而不是被脚部 streamer 的子集限制。

工作内容：

- 新增 `BodyLandmarkStreamer.cs`。
- 从 Meta FullBody skeleton 获取完整关节数据。
- 输出 `Body root`。
- 输出 `Body joints_pose`。
- 每个关节包含世界位置和世界旋转。
- 处理 Body 更新事件，避免递归刷新导致程序崩溃。
- 通过 Unity 日志确认每帧有效关节数量。
- 在 `CONNECTIONS.md` 中记录全身数据协议。

全身数据格式：

```
Body root:, x, y, z, qx, qy, qz, qw
Body joints_pose:, [x, y, z, qx, qy, qz, qw] * 84
```

每个关节 pose 的含义：

```
x, y, z      世界位置
qx, qy, qz   四元数向量部分
qw           四元数标量部分
```

Unity 日志中需要关注：

```
Streaming full body
Valid joints: 84/84
High confidence: True
```

设计注意点：

- `BodyLandmarkStreamer` 应该作为全身数据的统一出口。
- 测试全身数据时，可以暂时移除或禁用 `FootLandmarkStreamer`。
- 不要在多个 GameObject 上重复挂多个全身 streamer，否则 Python 端会收到重复帧。
- 如果只需要全身数据，后续机器人控制不应依赖旧的手部/脚部单独协议。

常见问题：

- 程序运行一会儿闪退时，检查是否在 `WhenBodyUpdated` 回调中又触发了 Body 数据刷新。
- 如果 Unity 日志显示 84/84 但 Python 没有更新，检查网络端口和协议是否一致。
- 如果左右侧混乱，先检查 Unity 端只保留一个全身 streamer。

检查点：

```
Quest / Unity -> BodyLandmarkStreamer -> Body root + Body joints_pose
```

5. 阶段 3：实时全身可视化与坐标检查

目标：在 Python 中实时检查全身数据的位置、方向和连续性。

这个阶段的作用是把 Unity 发出的 84 个关节从抽象数字变成可观察的骨架。可视化不是最终目标，但它是后续机器人映射前最重要的调试工具。

工作内容：

- 扩展 `scripts/visualizer.py`。
- 增加对以下消息的解析：

```
Body root
Body joints
Body joints_pose
```

- 将 Unity 坐标转换到 Python 可视化坐标。
- 绘制完整身体骨架连线。
- 隐藏容易造成误解的 root 点，避免视觉上出现“三条腿”。
- 增加头部可视化：
  - 头部关节点小红点。
  - 头部方向轴。
  - 头部立方体线框。
- 调整头部立方体的位置和尺寸，使其更容易和颈椎区分。

坐标转换目的：

```
Unity/Quest 坐标需要转换成 Python 可视化中更直观的右手坐标。
否则前后、左右或高度方向可能看起来不符合直觉。
```

头部可视化设计：

- 小红点表示真实头部关节点。
- 立方体表示头部大致体块。
- 方向轴表示头部旋转。
- 立方体略高于头部关节点，便于和颈椎区分。

典型可视化命令：

```
python scripts/visualizer.py --protocol tcp --host localhost --port 8000 --show-fingers
```

需要做的检查动作：

- 原地站立几秒。
- 左右摇头。
- 点头。
- 抬左手。
- 抬右手。
- 抬左腿。
- 抬右腿。
- 身体轻微转向。

观察重点：

- 头部方向轴是否随摇头变化。
- 左右手是否没有镜像反转。
- 左右脚是否没有交换。
- 肩、肘、腕是否连线合理。
- 髋、膝、踝是否连线合理。
- 身体没有明显跳变。

常见问题：

- 出现“三条腿”通常是 root 到骨盆的连线造成视觉误解，可以隐藏 root 或移除 root 连线。
- 头部不明显时，可以增加方向轴、头部方块或颜色标记。
- 可视化抖动不一定意味着网络错误，也可能是 Body Tracking 推断本身不稳定。

检查点：

```
Python 实时显示全身骨架，头部方向随摇头变化。
```

6. 阶段 4：全身数据录制

目标：将实时全身关节数据保存下来，作为后续机器人映射和模拟器调试的数据源。

这个阶段的核心是获得可重复使用的数据。实时数据每次都不一样，而录制数据可以反复回放、反复调试坐标转换、学习输入特征和机器人控制逻辑。

工作内容：

- 新增录制脚本：

```
scripts/record_body_stream.py
```

- 支持 TCP/UDP 接收。
- 支持指定录制时长。
- 支持 Ctrl+C 停止后保存。
- 保存为 `.npz` 文件。
- 保存时间戳、root pose 和所有关节 pose。
- 在没有收到完整帧时避免写出空文件。

输出数据格式：

```
timestamps       [N]
root_poses       [N, 7]
joint_poses      [N, 84, 7]
joint_positions  optional
joint_count
metadata
```

字段说明：

```
timestamps       每帧相对录制开始的时间
root_poses       每帧 root 的 position + quaternion
joint_poses      每帧 84 个关节的 position + quaternion
joint_positions  如果收到无旋转版本 joints，则可额外保存
metadata         协议、端口、关节数量等录制信息
```

典型命令：

```
adb reverse tcp:8000 tcp:8000
python scripts/record_body_stream.py --protocol tcp --host localhost --port 8000 --duration 10
```

指定输出文件：

```
python scripts/record_body_stream.py --protocol tcp --host localhost --port 8000 --duration 10 -o recordings/stand_01.npz
```

建议录制的数据集：

```
stand_01.npz       原地站立
head_hand_01.npz   转头和抬手
leg_01.npz         抬腿和屈膝
turn_01.npz        身体轻微转向
mixed_01.npz       头、手、腿组合动作
```

录制时注意：

- 每段动作开始前先静止 2-3 秒，方便后续标定初始姿态。
- 动作不要一开始就太快，先保证数据连续。
- 每段动作最好控制在 10-20 秒，便于调试。
- 文件名应反映动作内容。

检查点：

```
Quest / Unity -> Python recorder -> recordings/*.npz
```

7. 阶段 5：录制时同步可视化

目标：作为阶段 4 的配套工具，在录制数据时同时看到实时骨架，避免录完后才发现动作或坐标有问题。

这个阶段不是新的算法主线，而是数据采集质量保障。它把实时检查和数据录制合并到同一个脚本中，这样录制期间可以立即发现 tracking 丢失、左右反转、头部方向异常或身体跳变。

工作内容：

- 新增脚本：

```
scripts/record_visualize_body_stream.py
```

- 同时执行：

```
实时接收 Body joints_pose
实时显示全身骨架
实时累计录制帧
退出时保存 .npz
```

典型命令：

```
adb reverse tcp:8000 tcp:8000
python scripts/record_visualize_body_stream.py --protocol tcp --host localhost --port 8000 --duration 10
```

指定输出文件：

```
python scripts/record_visualize_body_stream.py --protocol tcp --host localhost --port 8000 --duration 10 -o recordings/head_
```

录制窗口中需要观察：

- 标题栏中的 recorded frames 是否增加。
- 骨架是否持续刷新。
- 头部立方体是否随头部移动。
- 方向轴是否随头部转动。
- 录制期间是否出现明显断帧。

适合使用该脚本的情况：

- 第一次录某类动作。
- 调整 Unity 端 streamer 之后。
- 改动 Python 坐标转换之后。
- 需要快速确认文件中确实有完整 body frames。

检查点：

```
边看边录，得到可回放的 recordings/*.npz。
```

8. 阶段 6：离线回放与数据质量检查

目标：对 `.npz` 录制文件进行离线检查，并能反复查看同一段动作。

离线回放是进入机器人控制前的重要步骤。机器人控制中的很多问题，例如左右反转、坐标轴错误、尺度不合适，最好先在离线人体骨架中确认，而不是接到模拟器后再排查。

工作内容：

- 新增脚本：

```
scripts/replay_body_recording.py
```

- 支持普通速度播放。
- 支持慢速播放。
- 支持循环播放。
- 支持跳帧快速检查。
- 使用与实时可视化一致的身体骨架、头部小点、头部立方体和方向轴。

典型命令：

```
python scripts/replay_body_recording.py recordings/test_01.npz --loop
```

慢速检查：

```
python scripts/replay_body_recording.py recordings/test_01.npz --speed 0.3 --loop
```

快速检查：

```
python scripts/replay_body_recording.py recordings/test_01.npz --frame-step 3
```

检查内容：

- 左右手是否正确。
- 左右脚是否正确。
- 头部方向轴是否随摇头变化。
- 身体骨架是否连续。
- 抬手、抬腿、转头动作是否正确回放。
- 静止站立时骨盆和脚部是否稳定。
- 四元数方向是否明显异常。

建议记录检查结果：

```
文件名
动作内容
是否左右正确
是否头部方向正确
是否有跳变
是否适合用于机器人映射
```

常见问题：

- 回放窗口不动时，检查 `.npz` 中是否有 `joint_poses`。
- 动作很快时，使用 `--speed 0.3`。
- 动作太长时，使用 `--frame-step` 快速扫一遍。
- 头部方向轴不对时，优先检查四元数坐标转换。

检查点：

```
recordings/*.npz -> replay_body_recording.py -> 离线全身骨架回放
```

9. 阶段 7：模拟训练阶段总体定义

目标：当 VR 录制数据已经可靠后，进入模拟训练阶段。在这个阶段中，不再把手写尺度缩放或手写规则作为主线，而是训练模型把 VR 获得的人体动作转换成符合机器人身体结构的参考姿态，并重新训练适配新 VR 设备和新机器人的 TWIST2-style 控制策略。

这个阶段的核心认识是：机器人不应该直接追踪人体关节坐标。人和机器人身体比例、关节数量、关节轴、运动范围和动力学能力都不同。学习模型要做的是理解人的动作意图，并输出机器人能够自然执行的姿态。控制策略要在模拟器中学习如何稳定跟踪这些参考姿态。

推荐系统结构：

```
HTS 84 joints recording / live stream
        ↓
预处理：坐标转换、root 对齐、速度计算、滤波
        ↓
学习式 retargeter
        ↓
机器人 qpos / robot reference motion
        ↓
重新训练的 TWIST2-style policy
        ↓
机器人运动
```

学习模型负责：

- 从 84 个人体关节中提取动作意图。
- 处理不同人的身高和身体比例。
- 处理 VR 设备穿戴差异。
- 输出机器人可达、连续、自然的参考姿态。
- 避免把人体不可执行动作直接交给机器人。

TWIST2-style policy 负责：

- 根据机器人当前 proprioception 执行参考动作。
- 生成低层关节控制。
- 保持动态稳定。
- 在模拟器和真实机器人上运行。

建议不要一开始训练：

```
HTS 84 joints -> robot motor torque/action
```

更稳妥的学习目标是：

```
HTS 84 joints -> robot reference qpos -> TWIST2-style policy
```

第一阶段可以参考 TWIST2 的 observation、reward 和训练结构，但由于 VR 设备和机器人可能都不同，最终需要在模拟器中重新训练 policy，使策略适配新的 reference 分布、新机器人模型和新的遥操作数据。

10. 阶段 8：学习数据集采集与组织

目标：建立用于训练 retargeter 的原始人体动作数据集。

每条原始样本建议保存：

```
timestamps
root_poses
joint_poses          [N, 84, 7]
joint_positions      [N, 84, 3]
metadata
calibration_id
user_height optional
```

建议数据目录：

```
datasets/hts_g1/
  recordings/raw/
  recordings/calibration/
  splits/train.txt
  splits/val.txt
  splits/test.txt
  metadata.json
```

建议先录制的动作：

- 静止站立。
- 左手上举。
- 右手上举。
- 双手前伸。
- 躯干轻微转动。
- 原地左右重心移动。
- 小幅下蹲。
- 原地踏步。
- 慢速走动。
- 转身。
- 简单操作动作。

每段动作建议：

- 开始前静止 2-3 秒。
- 动作速度先慢后快。
- 单段 10-20 秒。
- 同一动作多录几次。
- 不同穿戴位置和不同用户都要记录 metadata。

原始数据质量过滤：

- 删除 tracking 丢失片段。
- 删除 root 高度突变片段。
- 删除关节位置跳变片段。
- 删除速度和加速度异常的片段。
- 保留动作开始前的静止校准窗口。
- 标记站立、上肢、下肢、转身等动作类别。

建议脚本：

```
check_hts_recording_quality.py
split_hts_dataset.py
summarize_hts_dataset.py
```

检查点：

```
多条 HTS recording 被整理成可训练、可验证、可复现实验的数据集。
```

11. 阶段 9：学习输入关键点与特征提取

目标：从 84 个全身关节中提取学习模型真正需要的关键点和时序特征。

机器人通常不能直接使用人体的 84 个关节点。人和机器人的骨架结构、自由度、比例和关节定义都不同。更合理的方法是先把人体骨架整理成学习模型可用的输入特征，例如 root-relative position、joint velocity、头部方向、双手位置、骨盆位置、脚部位置等。

建议提取：

```
Pelvis / Root
Head
LeftShoulder
RightShoulder
LeftElbow
RightElbow
LeftWrist
RightWrist
LeftHip
RightHip
LeftKnee
RightKnee
LeftAnkle
RightAnkle
```

建议脚本：

```
scripts/extract_learning_features.py
```

输入：

```
recordings/*.npz
```

输出：

```
learning_features/*.npz
```

建议输出内容：

```
timestamps
pelvis_position
pelvis_quat
head_position
head_quat
left_shoulder_position
right_shoulder_position
left_elbow_position
right_elbow_position
left_wrist_position
right_wrist_position
left_hip_position
right_hip_position
left_knee_position
right_knee_position
left_ankle_position
right_ankle_position
root_relative_joint_positions
joint_velocities
body_direction
```

提取逻辑需要包含：

- 从 `joint_poses[:, joint_index, :]` 中取出指定关节。
- 保留原始时间戳。
- 保留必要的 position 和 quaternion。
- 输出一个更小、更适合学习训练的数据文件。
- 可选地输出 JSON metadata，记录每个 key 对应的 Meta joint index。

建议增加一个关键点可视化脚本：

```
scripts/visualize_learning_features.py
```

该脚本只显示学习模型重点使用的人体关键点：

```
head
pelvis
left wrist
right wrist
left elbow
right elbow
left ankle
right ankle
left knee
right knee
```

检查内容：

- 提取出来的关键点和原始骨架位置一致。
- 左右关键点没有交换。
- 头部四元数保留正确。
- 时间戳数量和原始录制一致。
- 输出文件比原始 84 关节文件更简单。

检查点：

```
recordings/*.npz -> extract_learning_features.py -> learning_features/*.npz
```

12. 阶段 10：学习输入坐标统一与归一化

目标：把 Quest/Unity 的人体坐标统一到模拟器和学习模型使用的坐标表示。

坐标转换是后续机器人控制中最容易出错的部分。即使人体骨架看起来正确，只要机器人模拟器坐标系和人体坐标系不一致，就会出现抬左手变成抬右手、向前变成向侧方、头部朝向错误等问题。

需要处理：

- 坐标轴方向。
- 左右是否反转。
- 高度方向。
- 初始朝向。
- 人体尺度和机器人尺度差异。
- 初始站姿对齐。

Unity/Quest 常用坐标：

```
x: 右
y: 上
z: 前
```

许多机器人模拟器常用坐标：

```
x: 前
y: 左
z: 上
```

建议脚本：

```
scripts/calibrate_hts_learning_coordinates.py
```

输入：

```
静止站立 recording
```

输出：

```
calibration/body_to_robot_calibration.json
```

标定文件建议包含：

```
axis_mapping
scale
origin_offset
initial_pelvis_position
initial_head_forward
user_height_estimate
robot_height
```

标定步骤：

- 读取一段静止站立数据。
- 取前几秒的 pelvis、head、left ankle、right ankle 平均值。
- 用脚踝或骨盆估计地面高度。
- 用 head 到 foot 的距离估计用户身高。
- 计算人体尺度到机器人尺度的缩放比例。
- 计算人体初始朝向到机器人初始朝向的旋转。
- 保存坐标转换参数。

验证动作：

- 静止站立。
- 抬左手。
- 抬右手。
- 向前伸手。
- 左右转头。

检查内容：

- 抬左手后机器人目标中的 left wrist 上升。
- 抬右手后机器人目标中的 right wrist 上升。
- 向前伸手后目标点沿机器人前向移动。
- 头部前向和机器人前向一致。
- 手脚目标不会因为尺度问题离机器人过远。

检查点：

```
人体关键点 -> 坐标统一和归一化 -> 学习模型输入特征
```

13. 阶段 11：训练模拟器与机器人模型接入

目标：选择并加载目标人形机器人模型，为后续 reference 生成、策略训练和 sim2real 做准备。

建议分清两个模拟器用途：

```
MuJoCo / Python: reference 回放、可视化调试、闭环验证
IsaacGym / legged_gym: 大规模并行 policy 训练
```

原因：

- MuJoCo 适合快速检查机器人模型、关节命名、reference 是否自然、实时闭环是否接通。
- IsaacGym / legged_gym 更适合重新训练 TWIST2-style policy，因为需要大量并行环境和长时间采样。
- 两者需要共享同一套机器人关节定义、动作维度、限位和 observation/reference 约定。

MuJoCo 侧需要准备：

- 一个可运行的人形机器人 MJCF/XML 模型。
- 能在 MuJoCo 中加载并显示模型。
- 能读取机器人关节状态。
- 能设置机器人关节目标或 actuator control。
- 能以固定 timestep 运行仿真。

Policy 训练侧需要准备：

- 与 TWIST2-style policy 匹配的机器人资产。
- 与机器人模型一致的 action space 和 joint order。
- 可批量加载 reference motion / mimic obs 的训练环境。
- reward、termination、domain randomization 和 observation history 配置。

建议脚本：

```
scripts/mujoco_load_robot.py
```

脚本需要做的事：

- 加载 MJCF/XML 模型。
- 打开 MuJoCo viewer。
- 打印机器人 joint names。
- 打印 actuator names。
- 打印 body/site names。
- 设置一个初始站姿。
- 让仿真稳定运行。

需要整理的信息：

```
机器人头部关节名称
机器人左肩/右肩关节名称
机器人左肘/右肘关节名称
机器人左腕/右腕或末端 site 名称
机器人骨盆/root body 名称
机器人左右脚 site 名称
关节角限制
执行器控制范围
```

检查内容：

- 机器人模型能正常显示。
- 初始姿态不会立刻倒下或爆炸。
- 手动设置关节角能看到对应身体部位运动。
- 机器人左右关节命名清楚。
- 找到手腕、头部、脚部的 site 或 body。

检查点：

```
MuJoCo 打开机器人模型，并能手动改变关节角。
```

14. 阶段 12：Teacher Label 与 Robot Reference 生成

目标：把已有 HTS 录制批量转换成可以训练模型的机器人姿态标签，并在模拟器中检查这些标签是否符合机器人结构。

teacher label 的作用不是最终控制机器人，而是为 learned retargeter 提供训练目标。初始 teacher/reference 可以来自模拟优化、可行性投影、人工筛选的 motion，或者临时使用已有工具生成的候选标签：

```
HTS 84 joints
        ↓
simulation optimization / feasibility projection / optional adapter
        ↓
robot feasibility projection
        ↓
robot qpos / TWIST2-style mimic obs
```

已有工具可以作为早期生成标签的辅助，但不应成为主线依赖。主线目标是构建自己的学习式人体到机器人姿态模型，并在模拟器中重新训练控制策略。

需要保存：

```
timestamps
input_hts_joint_poses
teacher_robot_qpos
teacher_mimic_obs
retarget_error
position_error
valid_mask
metadata
```

建议标签目录：

```
datasets/hts_g1/
  labels/teacher_qpos/
  labels/mimic_obs/
  labels/quality_reports/
```

质量过滤规则：

- 删除 tracking 丢失片段。
- 删除 root 高度突变片段。
- 删除 teacher retargeting error 过大的片段。
- 删除 G1 关节超限片段。
- 删除速度或加速度不连续片段。
- 标记站立、上肢、下肢、转身等动作类别。

建议脚本：

```
generate_hts_robot_teacher_labels.py
check_teacher_label_quality.py
view_robot_reference.py
```

robot reference 生成原则：

- 使用模拟优化、可行性投影或 teacher 模型生成机器人关节角。
- 不直接把人体关节点位置当作机器人关节角。
- 先生成上半身 reference，再扩展到全身 reference。
- 加入关节角限制和速度限制。
- 对 reference 做速度、加速度和平滑性约束。
- 对机器人不可达或动力学上不稳定的目标做过滤或降权。

teacher/reference 至少应覆盖的动作意图：

```
head orientation / yaw
left and right hand motion
pelvis height and yaw
left and right foot contact or stepping cue
upper body lean
```

离线生成流程：

```
recordings/*.npz
        ↓
extract_learning_features.py
        ↓
generate_hts_robot_teacher_labels.py
        ↓
check_teacher_label_quality.py
        ↓
robot reference motion / mimic obs
```

质量检查顺序：

- 先检查静止站立 reference 是否自然。
- 再检查上肢动作是否左右正确、方向正确。
- 再检查骨盆高度、躯干方向和脚部接触是否合理。
- 再检查连续动作中的速度和加速度是否平滑。
- 最后检查完整片段能否被 policy 训练环境稳定加载。

检查内容：

- 头部方向是否正确。
- 左右手是否没有交换。
- 上肢、骨盆和脚部 reference 是否在机器人可达范围内。
- reference 速度和加速度是否平滑。
- 关节角是否超限。
- 机器人是否出现突然抖动或穿模。

检查点：

```
recordings/*.npz -> teacher labels / robot reference -> 模拟器中可播放、可训练的机器人参考动作
```

15. 阶段 13：Learned Retargeter 训练

目标：训练一个模型，把 HTS 84 关节动作序列转换为机器人参考姿态。这个模型是人体动作和 TWIST2-style policy 之间的中间层，它不直接控制电机，而是输出机器人可执行、连续、自然的 reference。

阶段输入：

```
HTS recordings
learning features
coordinate calibration
teacher robot qpos / mimic obs
valid mask
metadata
```

阶段输出：

```
retargeter checkpoint
normalization statistics
predicted robot qpos
predicted mimic obs
retargeting quality report
```

推荐输入：

```
root-relative joint positions
joint velocities
root height
yaw-normalized body direction
optional: raw joint quaternions
optional: confidence / valid mask
```

推荐输出：

```
robot qpos: root xyz + root quat + robot dof
```

也可以输出 TWIST2 直接使用的 35D mimic obs：

```
base_vel_local_xy
root_height
roll_pitch
yaw_ang_vel_local
29 robot joints
```

建议优先输出 robot qpos。原因是 qpos 更容易可视化、检查和做物理约束；之后再由 qpos 转换成 TWIST2-style policy 需要的 mimic obs。

推荐模型结构：

```
HTS frame feature
        ↓
MLP encoder
        ↓
GRU / Temporal Conv / Transformer
        ↓
MLP decoder
        ↓
robot qpos
```

实现上可以先用 GRU，因为它实现简单、对短时动作连续性友好、训练成本低。后续如果需要更长时间上下文，再替换为 Temporal Conv 或 Transformer。

数据组织：

```
train split: 用于训练模型参数
validation split: 用于选择 checkpoint 和调参
test split: 只用于最终评估，不参与训练和调参
```

训练样本建议使用滑动窗口，而不是单帧输入：

```
past HTS features [t-k ... t]
current HTS features [t]
optional future short horizon [t+1 ... t+h]
        ↓
robot reference at t or short horizon
```

这样模型可以利用动作速度、方向变化和短时上下文，减少输出抖动。

基础 loss：

```
qpos reconstruction loss
joint position loss
root height loss
root orientation loss
velocity smoothness loss
joint limit penalty
```

机器人可行性 loss：

```
dof velocity loss
dof acceleration loss
foot sliding penalty
foot contact consistency
roll / pitch stability penalty
left-right symmetry regularization
```

训练流程：

```
加载 HTS recording
        ↓
加载 teacher robot qpos label
        ↓
构建滑动窗口序列
        ↓
训练 HTS -> robot qpos 模型
        ↓
离线生成 predicted qpos
        ↓
MuJoCo 播放检查
```

推理流程：

```
HTS live frame / recording frame
        ↓
same preprocessing and normalization as training
        ↓
retargeter inference
        ↓
robot qpos
        ↓
qpos to mimic obs
        ↓
TWIST2-style policy
```

验证指标：

- qpos 是否连续。
- 关节是否频繁到限位。
- root 高度是否稳定。
- 脚底是否明显滑动。
- 上肢动作是否跟随人体意图。
- 下肢动作是否保持平衡。
- 相比 teacher reference 是否更平滑、更自然。

检查点：

```
HTS recording -> learned retargeter -> robot qpos -> MuJoCo reference playback
```

16. 阶段 14：TWIST2-style Policy 重新训练

目标：在模拟器中重新训练适配当前 VR 数据和当前机器人的 TWIST2-style 控制策略。

训练输入来自 learned retargeter 或 teacher reference：

```
HTS recording / live stream
        ↓
learned retargeter
        ↓
robot reference qpos / mimic obs
        ↓
simulation rollout
        ↓
policy observation: mimic obs + proprioception + history/future reference
        ↓
TWIST2-style policy
        ↓
robot action / PD target
```

这个阶段可以套用 TWIST2 的训练框架：

```
IsaacGym / legged_gym
PPO / imitation learning
motion reference tracking reward
domain randomization
```

需要根据新机器人重新确认：

```
robot XML / URDF
joint order
actuator order
default dof position
joint limits
action scale
observation size
mimic obs definition
```

训练奖励建议包含：

```
reference joint tracking
root height tracking
root orientation tracking
foot contact stability
torque penalty
joint velocity penalty
action smoothness
fall penalty
```

通过标准：

- policy 能在模拟器中稳定跟踪 learned reference。
- 不同录制动作都能稳定运行。
- 机器人不会因为上肢动作破坏平衡。
- 下肢和 root 动作自然且不频繁触碰关节限制。
- 加入噪声、延迟和机器人参数扰动后仍然稳定。

检查点：

```
HTS recording -> learned retargeter -> retrained TWIST2-style policy -> stable simulation
```

17. 阶段 15：离线 Recording 模拟闭环验证

目标：使用录制好的 HTS recording，在模拟器中验证 learned retargeter 和重新训练 policy 的完整离线闭环。

离线闭环结构：

```
HTS recording
        ↓
learning feature extraction + normalization
        ↓
learned retargeter
        ↓
robot reference qpos / mimic obs
        ↓
retrained TWIST2-style policy
        ↓
MuJoCo / Isaac simulation
```

这个阶段的重点是排除实时网络和 VR 在线输入的影响，只验证模型和策略本身。

需要检查：

- recording 能否完整播放。
- learned retargeter 输出是否连续。
- policy 是否稳定跟踪 reference。
- 机器人是否长时间不摔倒。
- root 高度、roll、pitch 是否稳定。
- 关节角、速度、力矩是否没有异常峰值。
- 上肢动作是否符合人体意图。
- 下肢动作是否自然并保持平衡。

建议记录：

```
recording name
retargeter checkpoint
policy checkpoint
simulation duration
fall count
tracking error
root height range
max joint velocity
max torque
```

检查点：

```
HTS recording -> learned retargeter -> retrained policy -> stable offline simulation
```

18. 阶段 16：实时 VR 到模拟器闭环验证

目标：在离线 recording 模拟闭环通过后，把输入替换成实时 VR stream，在模拟器中验证 learned retargeter 和重新训练 policy 的在线闭环。

实时系统：

```
Quest / VR device
        ↓
HTS live socket stream
        ↓
online preprocessing buffer
        ↓
learned retargeter inference
        ↓
robot reference qpos / mimic obs
        ↓
retrained TWIST2-style policy
        ↓
MuJoCo / Isaac simulation
```

实时推理需要处理：

- 固定 FPS。
- 滑动窗口缓存。
- 输入归一化。
- 输出平滑。
- 丢帧补偿。
- 延迟估计。
- 断连保护。
- 急停和安全默认姿态。

建议频率：

```
VR input: 30-60 Hz
learned retargeter: 30-60 Hz
TWIST2 policy: 100 Hz
MuJoCo simulation: 1000 Hz
```

实时系统的安全策略：

- 初期只开上肢。
- 再加入躯干。
- 最后再加入下肢。
- 所有输出先限幅。
- VR 数据丢失时回到默认姿态。
- 模型输出异常时冻结上一帧或进入安全姿态。

检查点：

```
VR live stream -> learned retargeter -> retrained policy -> stable robot simulation
```

19. 阶段 17：Sim2Real 准备与真实机器人分阶段部署

目标：当模拟器中的 learned retargeter 和重新训练 policy 已经完全跑通后，再进入 sim2real。

进入 sim2real 前必须满足：

```
离线 recording 可以稳定驱动模拟器。
实时 VR 输入可以稳定驱动模拟器。
policy 在扰动环境中长时间不摔倒。
关节角、速度、力矩没有异常峰值。
网络延迟和丢帧保护已经验证。
```

真实机器人部署顺序：

```
1. 只回放默认站姿和安全姿态。
2. 只接 learned retargeter，不发送真实 motor command。
3. 只控制上肢小幅动作。
4. 加入躯干小幅动作。
5. 加入下肢或移动动作。
6. 扩展到完整 VR 遥操作。
```

真实机器人安全要求：

```
physical emergency stop
software emergency stop
joint limit clamp
velocity clamp
torque/current monitoring
VR input timeout
model output sanity check
safe default pose recovery
```

检查点：

```
simulation-proven learned teleoperation policy -> real robot safe deployment
```

20. 阶段 18：机器人视觉回传到 VR

目标：把机器人相机画面显示到 Quest 中。

视觉回传是闭环遥操作的另一半。用户不仅要控制机器人，还需要通过 VR 设备看到机器人观察到的环境。

最小版本：

```
机器人相机
        ↓
PC 接收视频流
        ↓
Unity Texture
        ↓
Quest 中显示在一个前方平面上
```

扩展版本：

```
双目相机 -> VR 双目显示
深度图   -> 点云/深度辅助显示
状态数据 -> VR HUD
```

建议顺序：

```
1. 单目 RGB 视频回传。
2. Unity 中显示为纹理。
3. 加入延迟测量。
4. 扩展双目或深度显示。
```

可能的视频来源：

- 真实机器人 RGB 相机。
- MuJoCo/Isaac/Gazebo 模拟器相机。
- ROS image topic。
- RTSP stream。
- WebRTC stream。
- 自定义 TCP/UDP 图像流。

Unity 端显示方式：

- 将视频帧更新到 Texture2D。
- 将纹理显示在 VR 前方固定平面。
- 后续可以改为曲面屏、双目显示或 passthrough overlay。
- 在画面旁边显示机器人状态 HUD。

需要测量：

```
视频帧率
端到端延迟
丢帧率
分辨率
Unity 解码开销
Quest 显示稳定性
```

检查点：

```
机器人相机画面 -> Unity Texture -> Quest VR 显示
```

21. 阶段 19：最终闭环 VR 遥操作

目标：用户在 VR 中看到机器人视角，同时用身体动作控制真实机器人，形成完整闭环遥操作系统。

闭环结构：

```
用户在 Quest 中看到机器人视角
        ↓
用户移动头部 / 手部 / 身体
        ↓
learned retargeter 生成机器人参考姿态
        ↓
重新训练的 TWIST2-style policy 生成机器人控制
        ↓
真实机器人执行动作
        ↓
机器人相机画面和状态回到 Quest
```

系统需要包含：

- 急停。
- 工作空间限制。
- 碰撞检测。
- 关节限位。
- 速度限制。
- 力矩或电流监控。
- 延迟监控。
- 数据丢失保护。
- 操作模式切换。
- 机器人状态显示。

建议操作模式：

```
观察模式：只显示机器人视觉，不发送动作控制。
头部控制模式：只控制机器人头部或相机云台。
上肢控制模式：控制头部和双手。
全身控制模式：加入躯干、腿部或移动底盘。
安全停止模式：立即停止控制输出。
```

基础任务设计：

- 用户左右转头，机器人视角跟随。
- 用户抬起左手，机器人左手抬起。
- 用户抬起右手，机器人右手抬起。
- 用户伸手靠近目标，机器人末端靠近目标。
- 用户在 VR 中根据机器人相机画面完成简单指向或抓取前置动作。

闭环系统需要记录：

```
VR body frame timestamp
retargeter output timestamp
robot command timestamp
robot state timestamp
camera frame timestamp
end-to-end latency
control frequency
video frequency
emergency stop events
```

检查点：

```
VR 视觉反馈 + learned retargeter + 重新训练 policy + 真实机器人执行 = 最终闭环遥操作
```

22. 阶段顺序总览

```
A. HTS 数据链路
阶段 0：原始 HTS 手部追踪验证
阶段 1：脚部和腿部关节扩展
阶段 2：全身 84 个关节输出
阶段 3：实时全身可视化与坐标检查
阶段 4：全身数据录制
阶段 5：录制时同步可视化
阶段 6：离线回放与数据质量检查

B. 模拟学习训练
阶段 7：模拟训练阶段总体定义
阶段 8：学习数据集采集与组织
阶段 9：学习输入关键点与特征提取
阶段 10：学习输入坐标统一与归一化
阶段 11：训练模拟器与机器人模型接入
阶段 12：Teacher Label 与 Robot Reference 生成
阶段 13：Learned Retargeter 训练
阶段 14：TWIST2-style Policy 重新训练
阶段 15：离线 Recording 模拟闭环验证
阶段 16：实时 VR 到模拟器闭环验证

C. Sim2Real 与最终闭环
阶段 17：Sim2Real 准备与真实机器人分阶段部署
阶段 18：机器人视觉回传到 VR
阶段 19：最终闭环 VR 遥操作
```

建议执行顺序：

```
先确认数据可靠。
再进入模拟训练阶段。
再构建学习数据集。
再提取学习模型输入特征，并做坐标统一和归一化。
再接入目标机器人模型和训练模拟器。
再生成 teacher label 或 robot reference。
再训练 learned retargeter。
再重新训练适配当前 VR 设备和机器人的 TWIST2-style policy。
再完成离线 recording 模拟闭环。
再完成实时 VR 输入下的模拟器闭环。
再进入 sim2real 和真实机器人分阶段部署。
再做机器人视觉回传。
最后完成真实机器人上的闭环 VR 遥操作。
```

23. 风险和注意事项

- Meta Body Tracking 是模型推断结果，不是高精度动作捕捉。
- 腿部和脚部数据可能在部分姿态下不稳定。
- 机器人不能直接使用人体关节点，需要先转换成符合机器人结构和动力学的 reference。
- 用户身体比例和机器人比例不同，需要做尺度归一化。
- 学习方法可以减少手写缩放和手写规则，但不能完全绕过人体和机器人比例差异。
- 学习模型不应该直接输出电机力矩作为第一阶段目标。
- 如果 teacher label 质量差，模型初期会继承 teacher 的缺陷。
- 如果重新训练 policy 时 reference 分布不稳定，policy 会学到不自然或不稳的行为。
- 数据集中如果动作类型太少，模型会在新动作上泛化差。
- 训练前需要严格区分 train、validation 和 test motion。
- 实时控制真实机器人前，需要先在模拟器中完成离线和实时闭环验证。
- 真实机器人控制需要急停、限位和速度限制。
- VR 视觉回传存在延迟，需要单独测量和优化。
- 网络抖动会影响实时控制，需要加入断连保护。
- 坐标系错误会导致机器人动作方向错误，需要单独做标定和可视化验证。
- 不可达或动力学上不稳定的人体目标需要过滤、降权或投影，不能直接让机器人追踪任意人体位置。